

# End-to-End DevOps CI/CD Pipeline on AWS

Rohit Gupta

Herovire Capstone, Batch 15

[github.com/rohitguptaangular/AWS-CICD-DEVOPS-CAPSTONE](https://github.com/rohitguptaangular/AWS-CICD-DEVOPS-CAPSTONE)

# Agenda

---

1. The problem and what I built
2. Architecture
3. Build and deploy pipeline (Jenkins)
4. Infrastructure as code (Terraform)
5. The design bug I found and fixed
6. Configuration management (Ansible)
7. Kubernetes and monitoring
8. Security
9. Evidence it runs
10. Cost control
11. GitOps path, and what I would do next

## What I built

---

A pipeline that takes my app from a `git push` to a running, monitored service on AWS, without any manual steps.

What it had to do:

- Containerize the app and build it automatically
- Create all AWS infrastructure from code, not the console
- Deploy to a Kubernetes cluster
- Monitor it and alert when something breaks
- Stay cheap and tear down cleanly

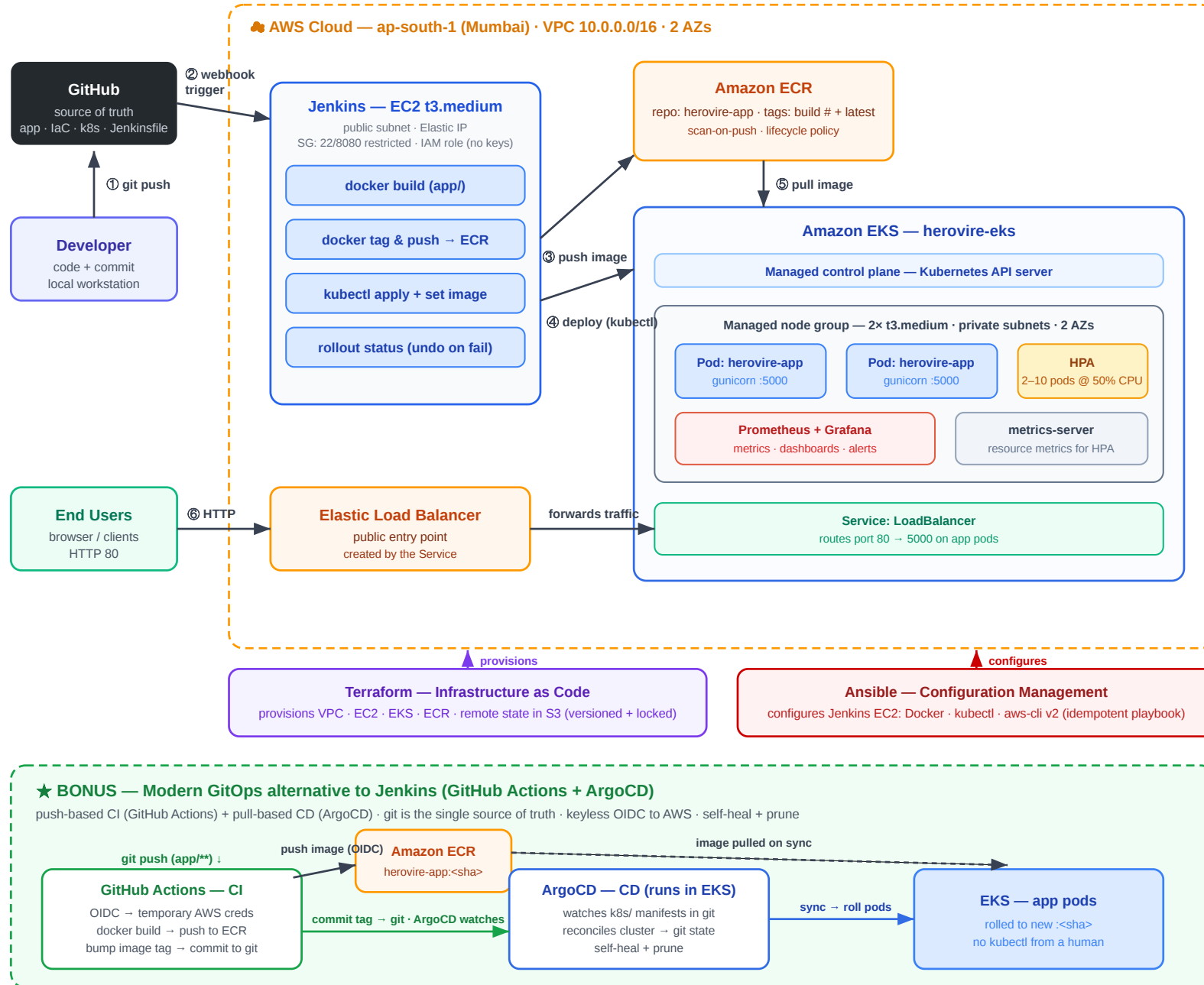
# Architecture

---

```
Developer
  | git push
  v
GitHub --> Jenkins (EC2) --> Docker build --> Amazon ECR
                                     |
Terraform (VPC, EKS, IAM, ECR) <-- provisions --|
Ansible (configures Jenkins host)               |
                                               v
                                   kubectl / rollout --> Amazon EKS
                                               |
Prometheus + Grafana <-- scrapes /metrics
```

# Herovire DevOps Capstone — End-to-End CI/CD Architecture on AWS

GitHub → Jenkins → Docker → ECR → EKS · provisioned with Terraform · configured with Ansible · monitored with Prometheus + Grafana



# Network layout

---

```
VPC 10.0.0.0/16 (ap-south-1, two AZs)
├── Public 10.0.1.0/24, 10.0.2.0/24
│   ├── Jenkins EC2 (t3.medium, static EIP)
│   └── Load balancer for the app
├── Private 10.0.11.0/24, 10.0.12.0/24
│   └── EKS worker nodes, no public IPs
├── Internet Gateway (public subnets)
└── NAT Gateway (one) (private subnets reach out)
```

Nodes are private, so the only way in is through the load balancer.  
One NAT gateway instead of one per AZ: a single point of failure,  
but it halves the NAT cost and is the right trade here.

## Tools I used

---

| Stage          | Tool                 | What it does                     |
|----------------|----------------------|----------------------------------|
| Build          | Docker + Jenkins     | Build the image, push to ECR     |
| Infrastructure | Terraform            | VPC, EKS, ECR, IAM, state in S3  |
| Configuration  | Ansible              | Sets up the Jenkins server       |
| Deploy         | kubectl and EKS      | Rolls the image onto the cluster |
| Monitoring     | Prometheus + Grafana | Metrics, dashboard, alerts       |

The app itself is a small Flask service with `/`, `/health` and `/metrics`.

## Build and deploy (Jenkins)

---

The `Jenkinsfile` pipeline runs:

```
Checkout -> Build -> ECR login -> Tag and Push -> Deploy -> Smoke Test
```

- Images go to ECR, tagged with the build number
- Deploy runs `kubect1 apply` then updates the image tag
- `kubect1 rollout status` gates the build, so it only passes if pods come up healthy
- If the smoke test fails, the pipeline rolls back with `kubect1 rollout undo`

## Infrastructure as code (Terraform)

---

Everything on AWS is defined in `terraform/`, split into two root modules with separate state:

| Module                  | Contains                       | Applied by           |
|-------------------------|--------------------------------|----------------------|
| <code>bootstrap/</code> | VPC, subnets, NAT, Jenkins EC2 | me, from a laptop    |
| <code>platform/</code>  | EKS, ECR, OIDC role            | the Jenkins pipeline |

State lives in a versioned S3 bucket, so the environment is reproducible.

`terraform plan` costs nothing, so I only ran `apply` when I was actually working.

## Configuration (Ansible)

---

Two layers on the Jenkins host:

- **EC2 user data** bootstraps it at first boot: Java 21, Jenkins, Docker, Terraform
- **Ansible** ( `configure-nodes.yml` ) then installs Docker, kubectl and the AWS CLI, and points kubeconfig at the EKS cluster

Ansible is idempotent, so re-running it converges to the same state. User data only ever runs once, on first boot, which is why the ongoing config lives in Ansible.

# Kubernetes

---

- Deployment with 2 replicas
- Service of type LoadBalancer, so the app is reachable publicly
- HPA to scale on CPU, 2 to 10 pods (needs metrics-server installed)
- Liveness and readiness probes on `/health`

# Monitoring

---

- Installed kube-prometheus-stack with Helm (Prometheus, Grafana, Alertmanager)
- The app uses `prometheus-flask-exporter` to expose `/metrics`
- A ServiceMonitor tells Prometheus to scrape it
- Grafana dashboard shows request rate: `rate(flask_http_request_total[1m])`
- Alert `AppPodDown` fires when the scrape target stops responding

# Security

---

**No static AWS credentials anywhere in the pipeline.**

| Identity         | Permissions                                         |
|------------------|-----------------------------------------------------|
| Jenkins EC2 role | <code>AdministratorAccess</code> , broad on purpose |
| EKS node role    | Worker, CNI, and ECR read only                      |
| GitHub Actions   | ECR power user, assumed via OIDC                    |

- Jenkins authenticates to AWS with an instance role, not access keys
- Worker nodes are private, with no public IPs
- Ingress is restricted: 22 and 8080 from my IP, 8080 from GitHub's webhook ranges, and 22 within the group itself for the Ansible stage

The Jenkins role is admin because it runs `terraform apply` across VPC, IAM, EKS and EC2. In production this would be a scoped provisioning role. I have called it out in the code rather than hidden it.

## Evidence it runs

---

Both Jenkins pipelines went green as real jobs, not from my laptop:

| Pipeline       | Result                                                                 |
|----------------|------------------------------------------------------------------------|
| herovire-app   | 6 stages, 57 seconds, image pushed and smoke test passed               |
| herovire-infra | Terraform applied, Ansible <code>ok=8 changed=3</code> , 2 nodes Ready |

- The infra job created the EKS cluster and node group from scratch
- A later run was a clean no-op: `0 added, 0 changed, 0 destroyed`
- Teardown removed 44 resources and returned the account to zero

Screenshots of every green build are in `docs/screenshots/`.

## Testing

---

- `tests/smoke_test.sh` checks that `/` and `/health` return 200
- `tests/infra_test.sh` checks nodes are Ready, pods Running, LB is up
- The smoke test runs as a Jenkins stage, so a bad deploy fails the build
- I ran the full cycle end to end: apply, test, destroy

## Bonus: GitOps with GitHub Actions and ArgoCD

---

I also built a second, more modern path and ran it live:

- GitHub Actions builds and pushes the image, authenticating with OIDC so no AWS keys are stored in GitHub
- It then updates the image tag in `k8s/deployment.yaml` and commits it
- ArgoCD watches the repo and syncs the cluster to match

One push took the app from v1 to v2 with no `kubectl` from me.

## Demo

---

1. App running as v1 on its LoadBalancer URL
2. Push a small change
3. GitHub Actions goes green
4. ArgoCD flips from OutOfSync to Synced, pods roll
5. Refresh, app is now v2
6. `kubectl scale --replicas=1` , ArgoCD puts it back to 2

## Problems I ran into

---

- My Mac builds arm64 images but the nodes are amd64, so pods failed to pull.  
Fixed by building with `--platform linux/amd64`
- Jenkins was installed but dead on every fresh deploy, because a failing `pip install` in user data stopped the script before Jenkins started
- Jenkins could not reach the cluster: it sits inside the VPC, so the EKS endpoint resolved to private IPs the security group did not allow
- Leftover LoadBalancers blocked the VPC from deleting, so I now delete the Services before running destroy
- Prometheus was scraping nothing because the Service was missing an `app` label

## Cost

---

- Running cost is roughly \$0.25 to \$0.30 per hour (EKS, 2 nodes, NAT, LB)
- Writing code and running `plan` is free, only `apply` costs anything
- Kept it small: 2 t3.medium nodes, 3 day metrics retention, one NAT gateway
- Destroyed everything after each session, so a full build and test cycle cost about \$0.25
- ECR lifecycle policy clears out untagged images

## What I would do next

---

- Scope the Jenkins IAM role down from `AdministratorAccess`
- Move the infra pipeline onto ephemeral runners so nothing long lived needs admin rights
- Add a Cluster Autoscaler, since today pods would sit `Pending` past 3 nodes
- Put the app behind an ALB with TLS instead of a Classic Load Balancer
- Rewrite `AppPodDown` as an `absent()` rule so it catches a true zero-pod state

## What I learned

---

- Infrastructure as code makes environments reproducible and disposable
- Terraform and Ansible solve different problems
- Push based delivery (Jenkins) and pull based delivery (ArgoCD) are both valid
- OIDC and short lived credentials are better than storing secrets
- A deploy is not finished until you can see that it is healthy
- Cost control has to be a habit, not something you fix at the end

## Summary

---

GitHub to Jenkins to Docker to ECR to Terraform to Ansible to EKS,  
with Prometheus and Grafana on top, plus a working GitOps path.

**One `git push` gets me a running, monitored app on Kubernetes.**

**Thank you. Questions?**